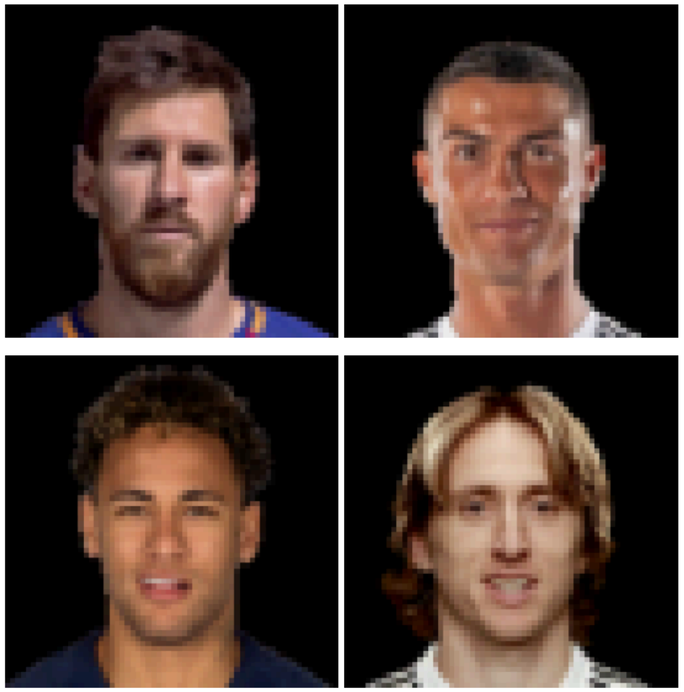
CLASE TEÓRICA · 60 MINUTOS

Autoencoders Variacionales

Aprender a crear comprimiendo

Ninguno de estos jugadores existe (otra vez)

REALES · SOFIFA.COM



GENERADAS · VAE



GENERADAS · DCGAN



La clase pasada llegamos a caras nítidas con un crítico. Hoy, otro camino: **comprimir y descomprimir**. Pregunta para el final: ¿por qué las del VAE se ven más borrosas?

Plan de la clase

Bloque	Tema	Minutos
1	Comprimir para entender	8
2	El autoencoder	8
3	¿Puede un autoencoder generar?	8
4	El VAE	14
5	VAE sobre FIFA	10
6	VAE contra GAN	6
7	Cierre	2

BLOQUE 1 · 8 MINUTOS

Comprimir para entender

Una cara son miles de números. Pero las caras no ocupan todo ese espacio.

Una cara son 12.288 números



$$64 \times 64 \times 3 = 12.288$$

Casi cualquier combinación de 12.288 valores es **ruido**. Las caras reales viven en un rincón chico de ese espacio.

> **INTUICIÓN**
>
> Con pocas «perillas» se describe casi todo: tono de piel, pelo, ángulo, sonrisa, color de camiseta. Reducir dimensiones es encontrar esas perillas.

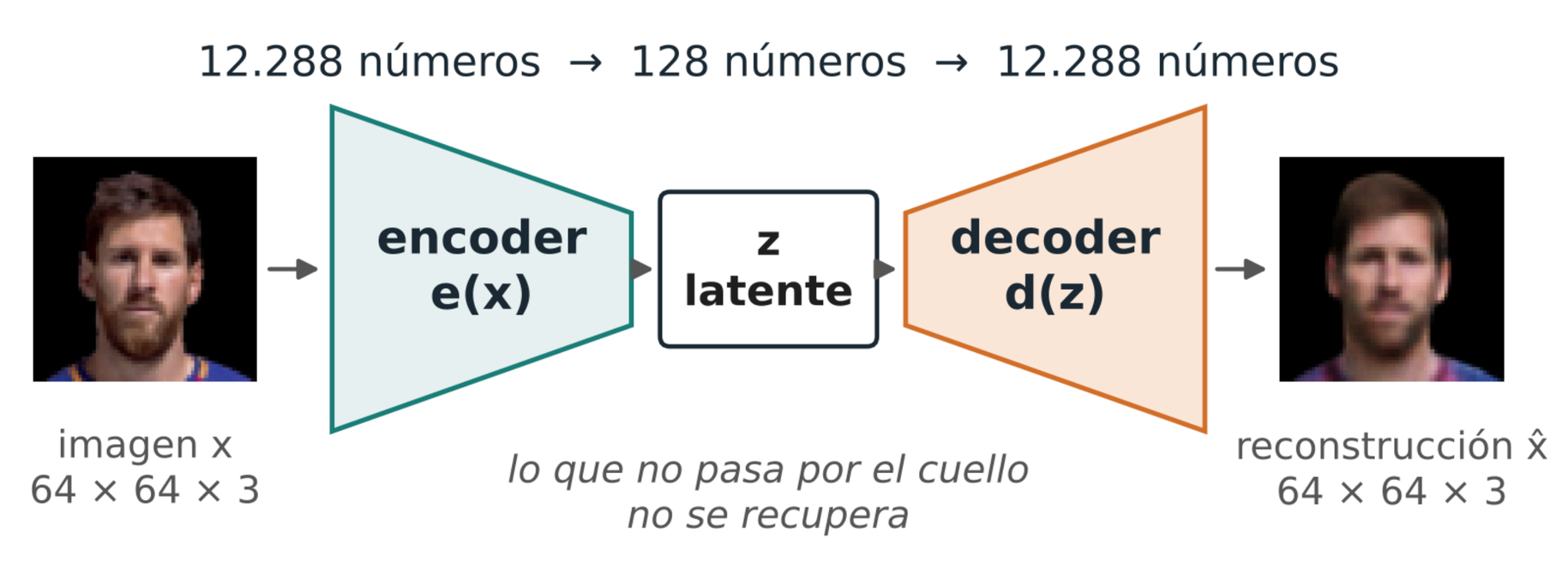
Selección

Quedarse con algunas características que ya existen.

Extracción

Crear características nuevas que resumen a las originales. Es lo que hace un encoder.

Encoder, espacio latente, decoder



Encoder: comprime la imagen a un vector chico.

Latente: donde viven los códigos z: las perillas.

Decoder: descomprime el vector a una imagen.

El objetivo: reconstruir lo mejor posible

$$(e^*, d^*) = \arg\min_{(e,d)} \epsilon(x, d(e(x)))$$

Buscamos el par encoder / decoder que **minimiza** el error ε entre la imagen original x y su reconstrucción d(e(x)).

> **INTUICIÓN**
>
> **El dato es su propia etiqueta.** No hace falta etiquetar nada: la respuesta correcta para cada cara es la misma cara.

Dos redes y un cuello de botella

Encoder e

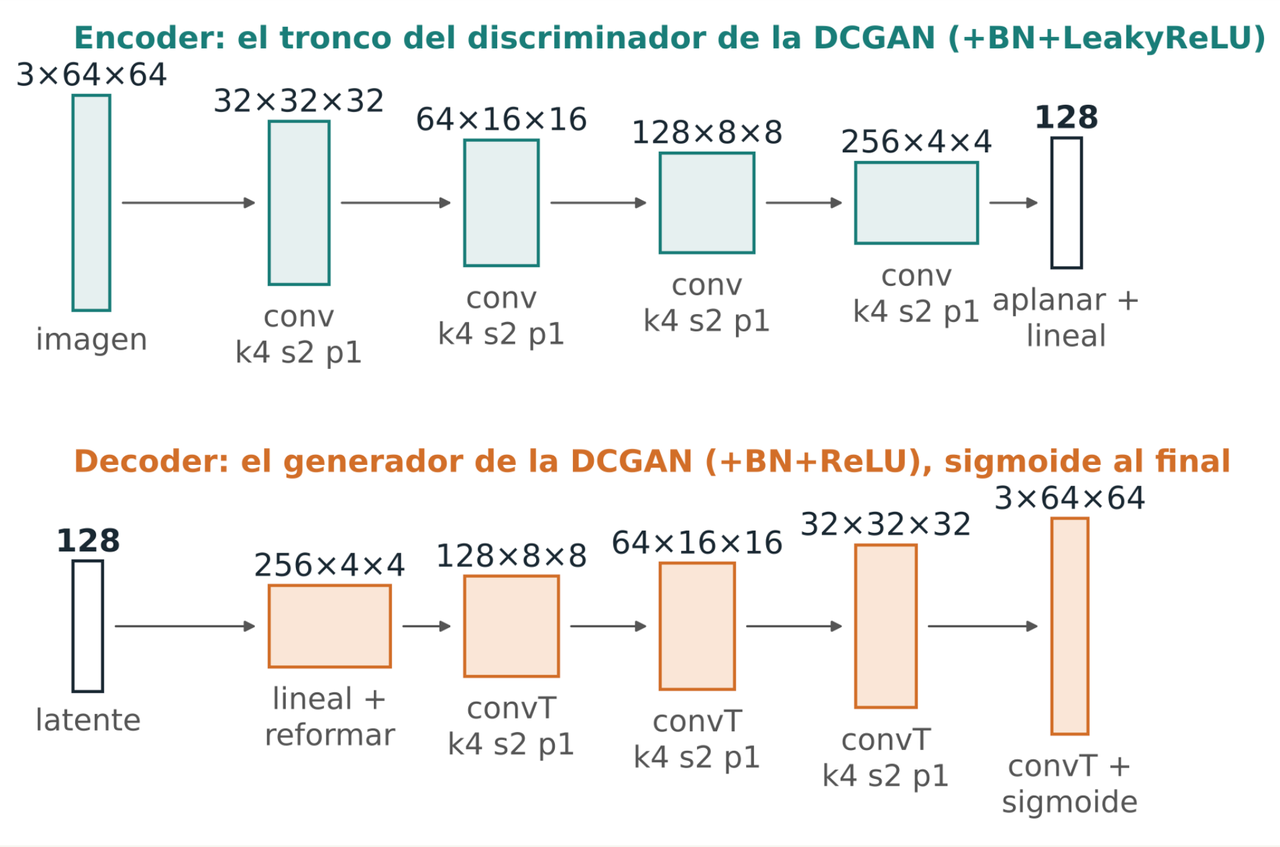
Una red que comprime la imagen a un vector de pocas dimensiones.

Decoder d

Una red que recibe ese vector y devuelve una imagen del tamaño original.

- Se entrenan **juntas**, de punta a punta, para minimizar el error de reconstrucción.
- El cuello (128 números contra 12.288) obliga a quedarse con **lo esencial**.
- Sin cuello, la red aprendería a copiar píxel por píxel y no aprendería nada.

La arquitectura sobre FIFA



Encoder

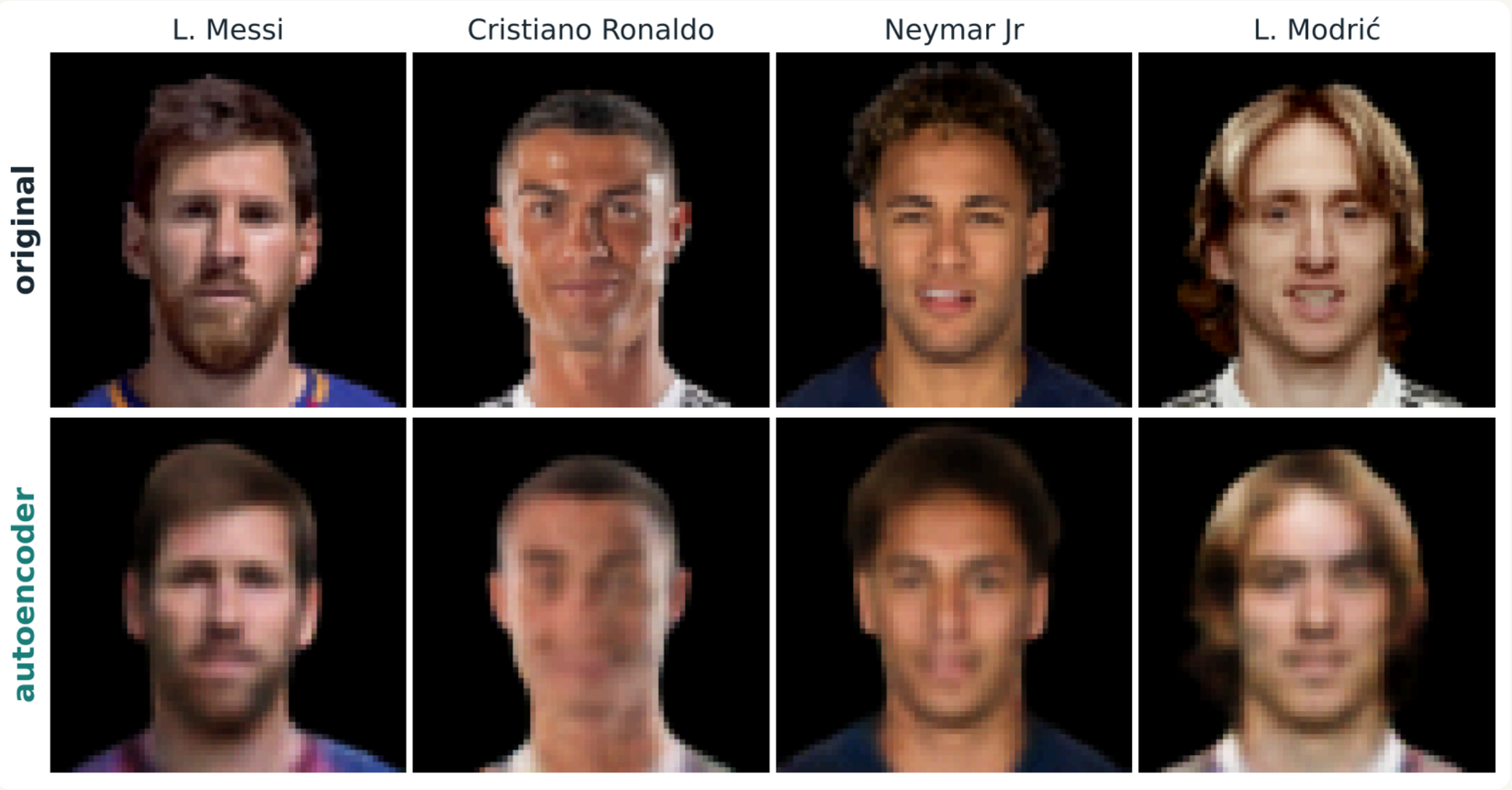
4 × conv k4 s2 p1 + BN + LeakyReLU. Cada paso divide la resolución por 2.

Decoder

El espejo: convolución transpuesta k4 s2 p1 + BN + ReLU, y una **sigmoide** al final: píxeles en [0, 1].

Son **los mismos bloques** del discriminador y del generador de la DCGAN, puestos uno detrás del otro.

Qué aprende



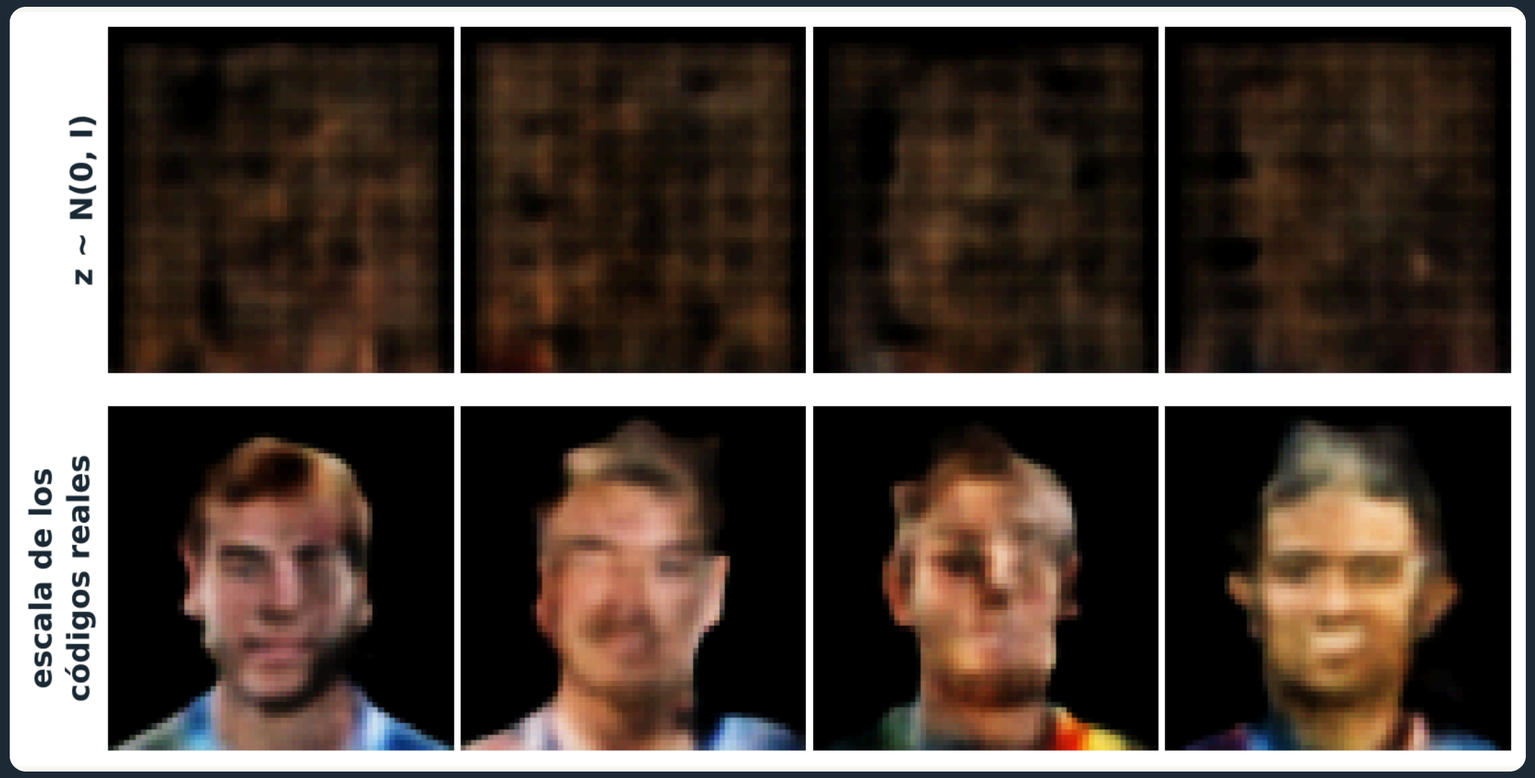
Sobrevive

Forma de la cara, peinado, colores, pose.

Se pierde

El detalle fino: barba, ojos, los rulos de Neymar.

La idea tentadora: sortear un z y decodificarlo



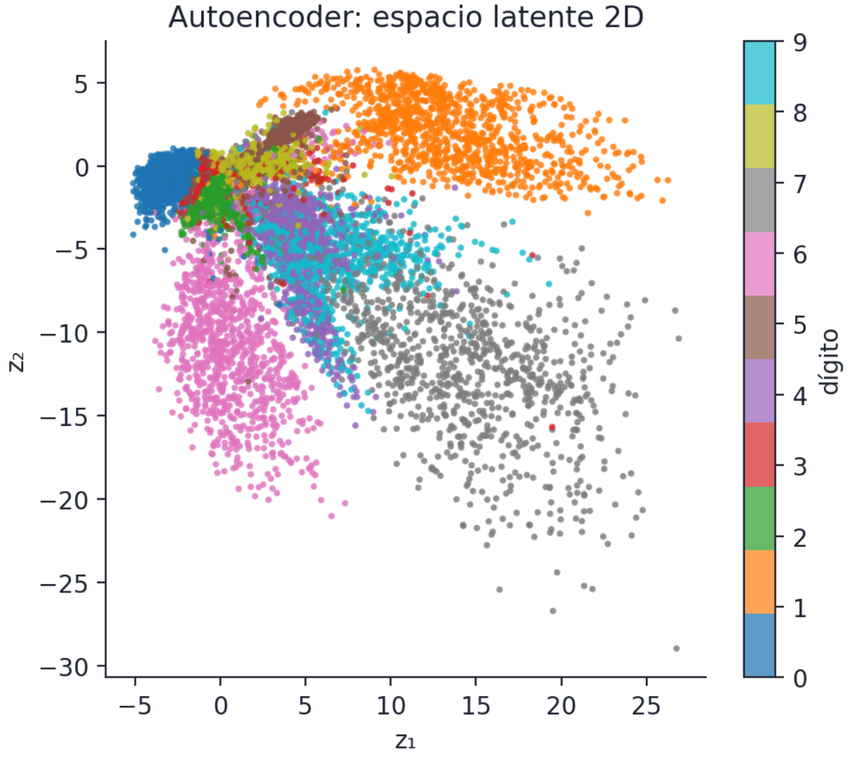
Ya tenemos un decoder, ¿no?

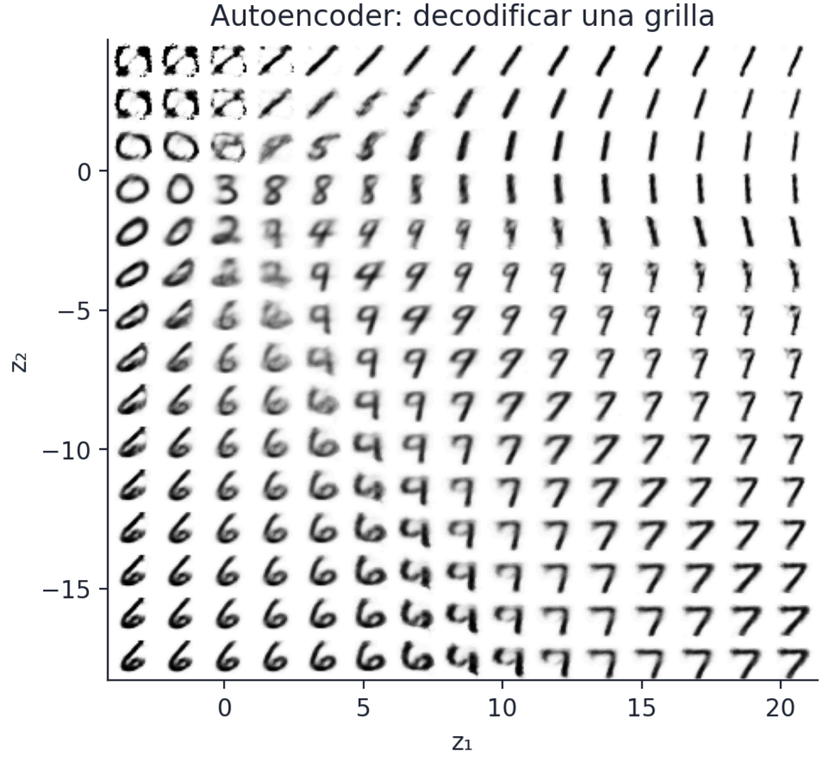
Arriba, z ~ N(0, I): **manchas**. Pero eso no es justo: los códigos del autoencoder no viven ahí.

Abajo, sorteamos con la media y el desvío de los códigos reales, en la misma región donde están las caras: **caras de pesadilla**.

No anda.

Por qué: islas y huecos



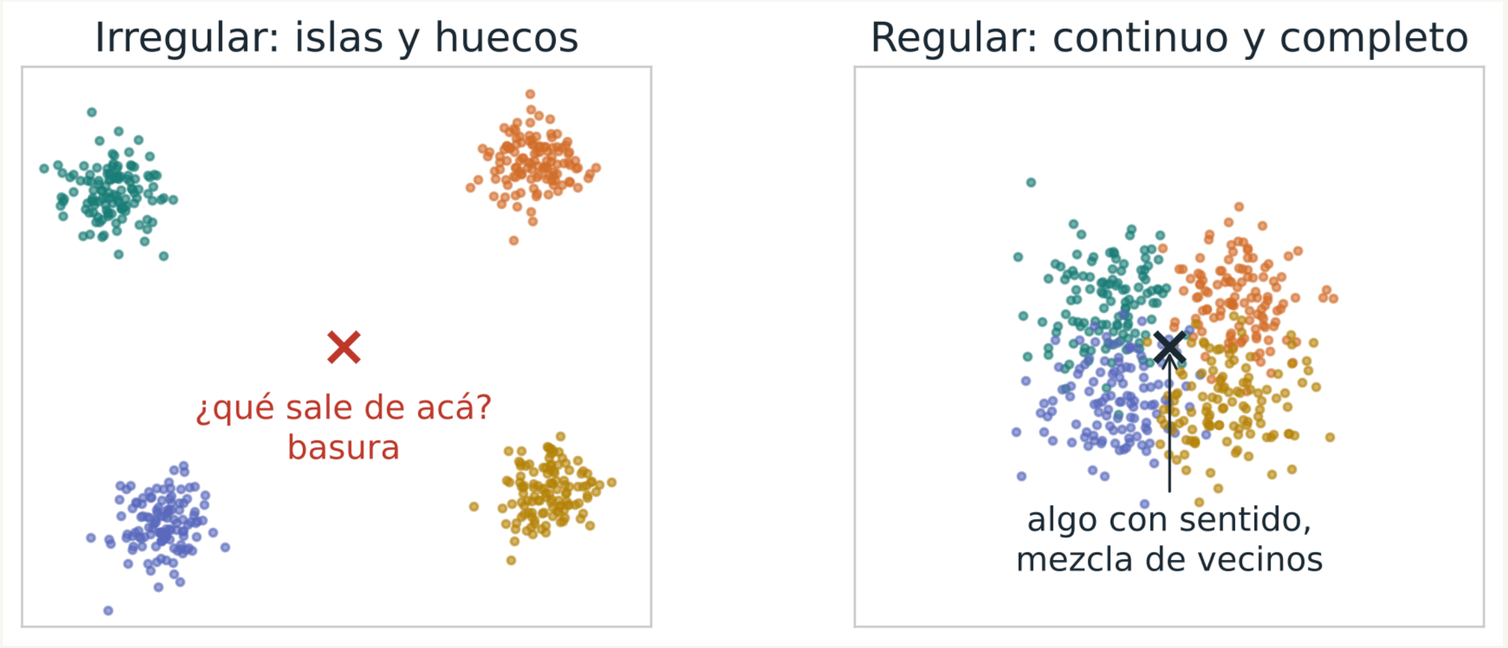


MNIST con latente de **2 dimensiones**: se puede *ver*.

Cada dígito es una **isla**, en cualquier lugar y escala.

Entre islas sale **basura**.

Lo que necesitamos: un espacio regular



Continuidad: dos puntos cercanos dan imágenes parecidas.

Completitud: cualquier punto razonable da algo con sentido.

Por qué no aparecen solas

Nadie se las pidió. La pérdida solo premia **reconstruir**: en ningún término dice «y que el espacio quede ordenado».

El caso extremo

Un encoder lo bastante potente podría mandar cada cara a **un número** de la recta real (Messi → 1, Cristiano → 2, …) y un decoder igual de potente reconstruiría cada una perfecto. Pérdida cero. Y el punto 1,5 no sería nada.

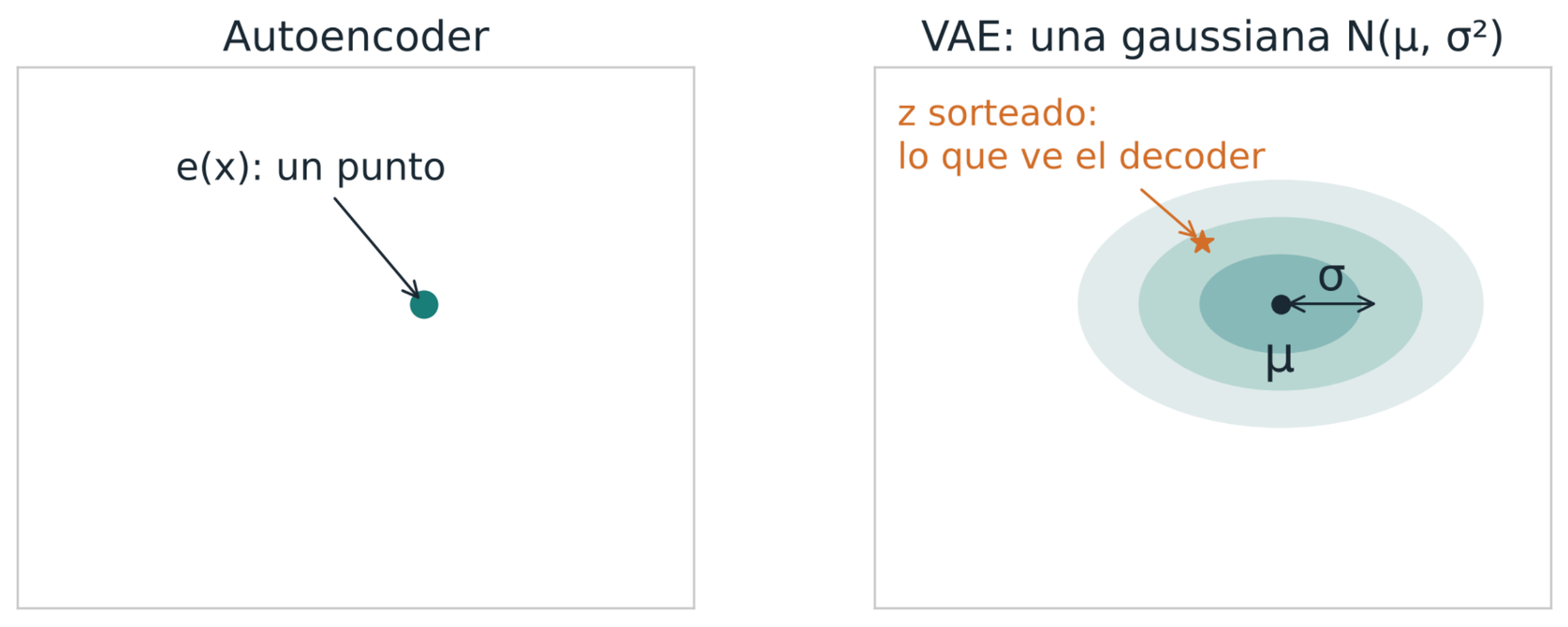
Sin regularización, la red aprovecha cualquier hueco para sobreajustar.

BLOQUE 4 · 14 MINUTOS

El VAE

Si queremos un espacio sin huecos, tenemos que pedirlo.

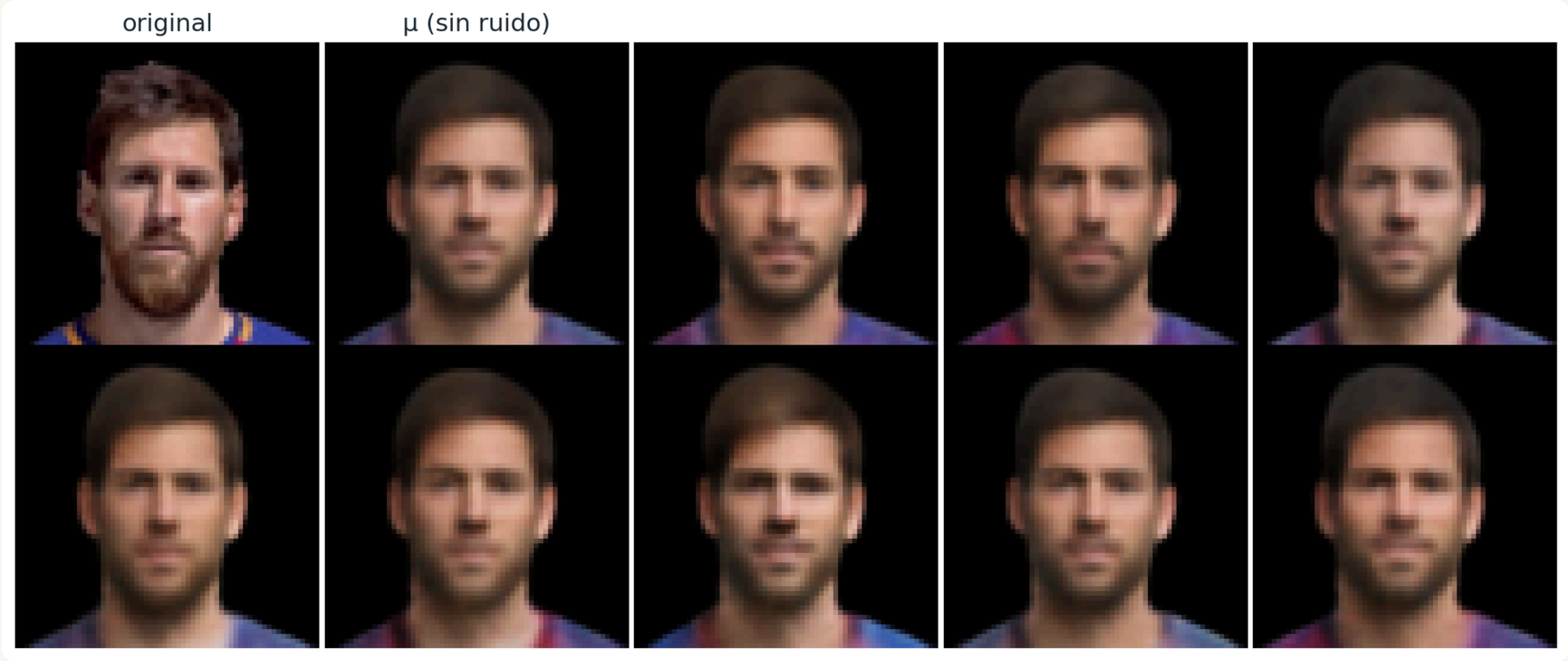
Codificar una nube, no un punto



$$q(z \mid x) = \mathcal{N}(\mu(x),\ \sigma^2(x))$$

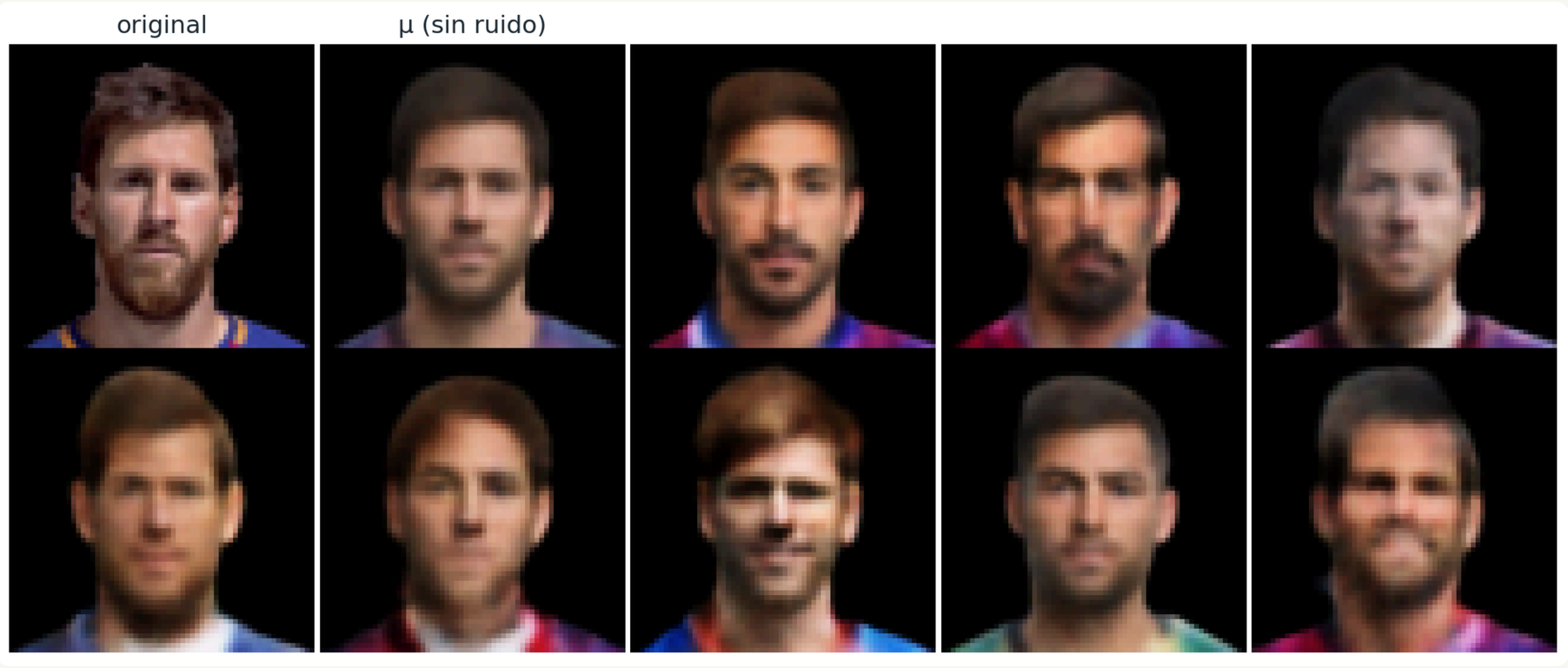
El encoder devuelve **una media μ y un desvío σ** por dimensión. Al decoder le llega un punto **sorteado** de esa nube.

La nube de Messi: muchos Messis



Codificamos a Messi y sorteamos **z = μ + σ·ε** diez veces. Todos los puntos de su nube decodifican a Messi, con variaciones mínimas.

Agrandando la nube (σ × 3)



Si nos alejamos más del centro, salimos de Messi hacia **sus vecinos**: otra barba, otro pelo, otra camiseta. Siempre una cara plausible, nunca basura.

Por qué la nube ayuda

Cada cara, una nube; las vecinas se pisan



El punto cambia en cada pasada: el decoder tiene que reconstruir bien **desde cualquier lugar de la nube**.

Aprende **regiones**, no puntos sueltos. Las vecinas **se pisan** y los huecos se llenan con mezclas.

> **INTUICIÓN**
> Como dibujar a Messi con el pulso temblando: el dibujo tiene que funcionar aunque la mano se corra un poco.

La trampa



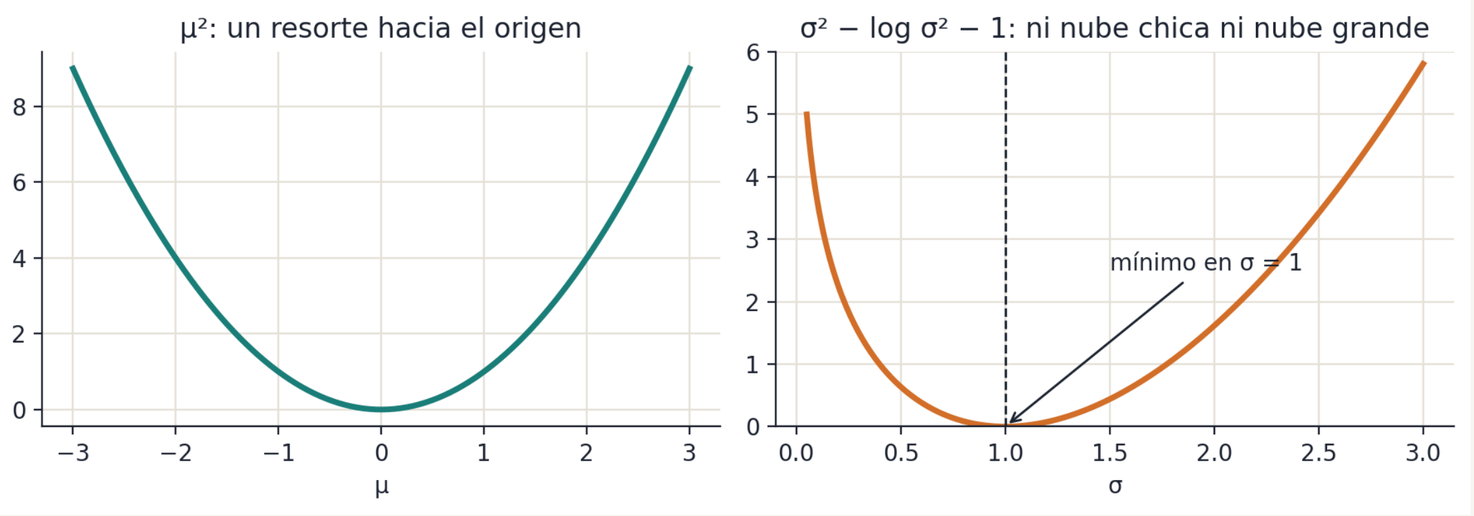
La red **achica las nubes** (σ → 0) o **las separa** (μ lejos): el ruido deja de molestar y vuelve a ser un autoencoder.

El resorte: la divergencia KL

$$\mathrm{KL} = \frac{1}{2}\sum_j \Big(\underbrace{\mu_j^2}_{\text{resorte al origen}} + \underbrace{\sigma_j^2 - \log \sigma_j^2 - 1}_{\text{ni chica ni grande}} \Big)$$

A cada nube se le pide que se parezca a **N(0, I)**. Definición general:

$$\mathrm{KL}(P \,\|\, Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$$



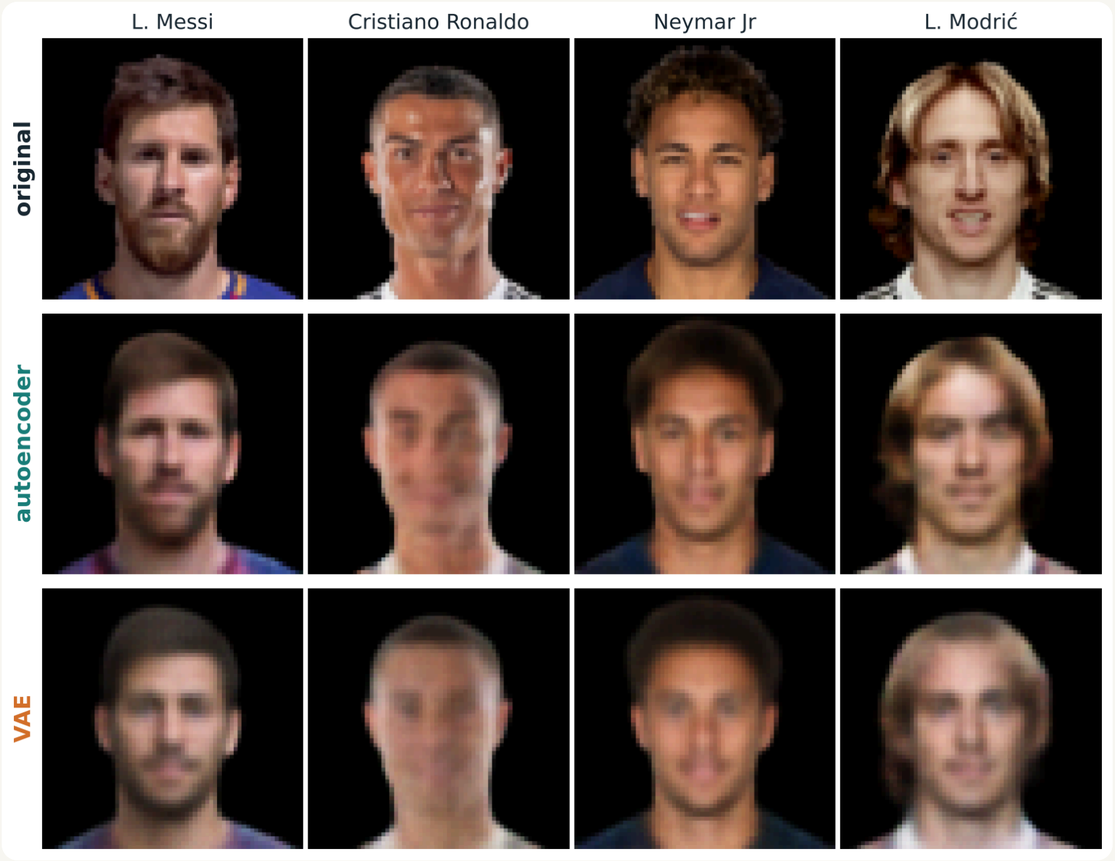
μ²

Resorte hacia el origen: impide **separar** las nubes.

σ² – log σ² – 1

Castiga la nube chica y la grande; vale 0 en σ = 1: impide **achicarlas**.

La pérdida completa: reconstrucción + KL



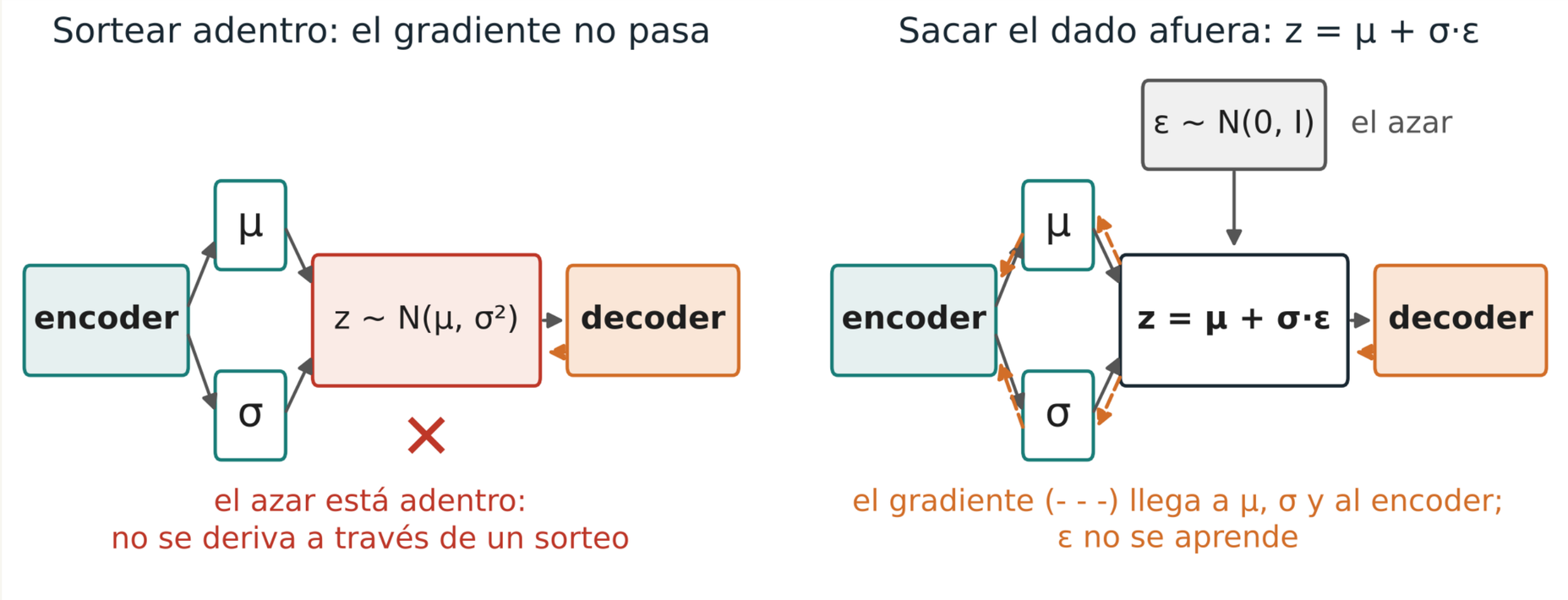
$$\mathcal{L}(x) = \underbrace{\mathrm{BCE}(x,\ \hat{x})}_{\text{reconstrucción}} + \underbrace{\mathrm{KL}(\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0, I))}_{\text{regularización}}$$

La **reconstrucción** quiere nubes chicas y separadas.

La **KL** quiere nubes estándar y encimadas.

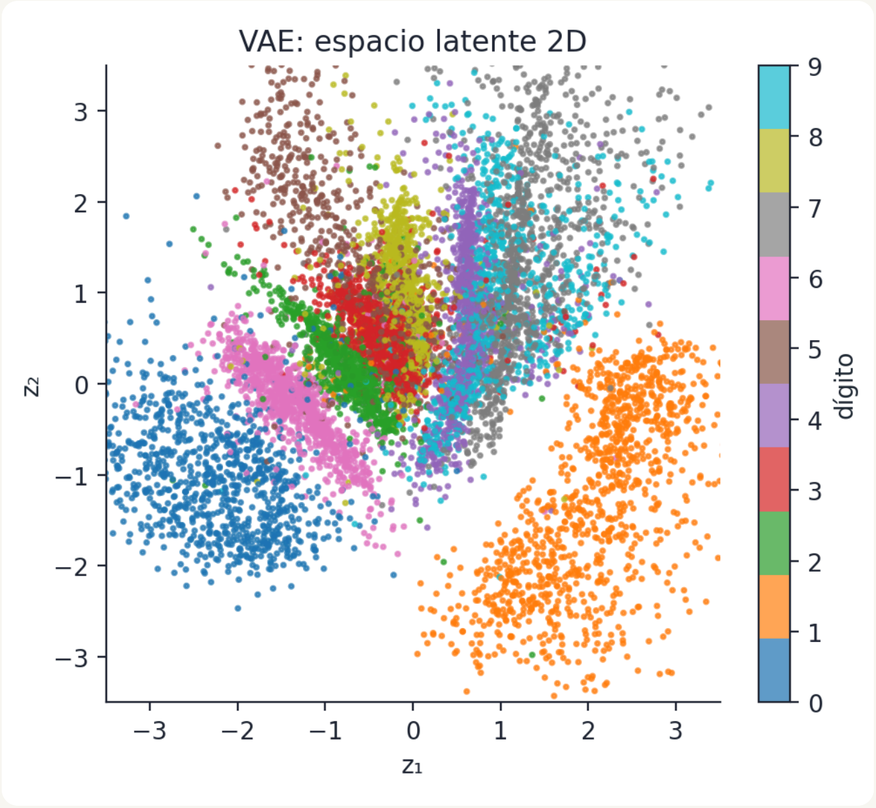
El VAE queda en el medio: reconstruye **un poco peor** que el autoencoder. Es el precio de un latente ordenado.

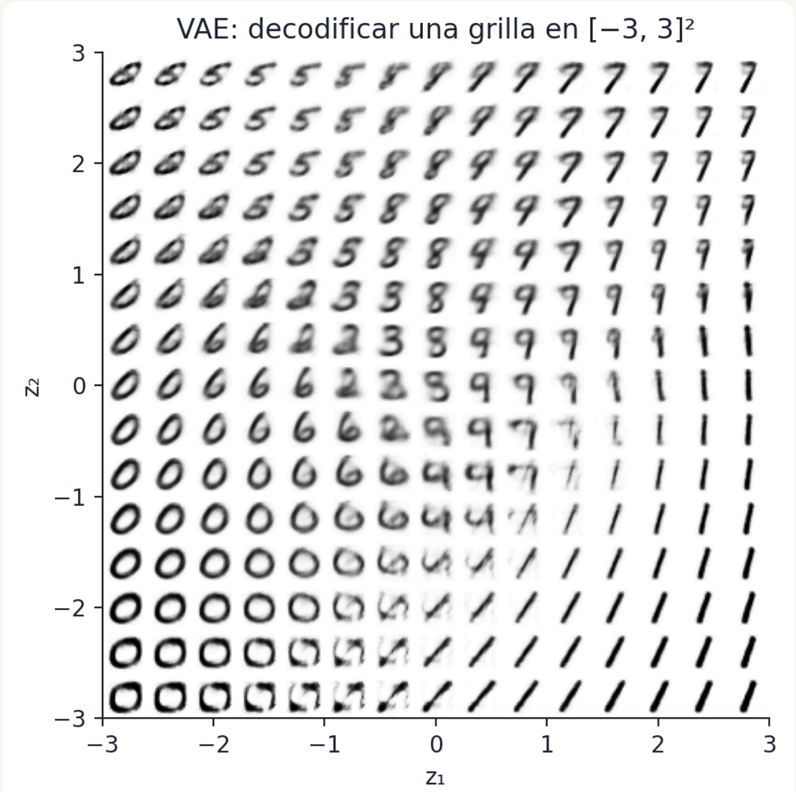
El truco de la reparametrización



Sacar el dado afuera. El azar queda en ε, que no se aprende; μ y σ quedan en el camino del gradiente.

Generar ahora es fácil





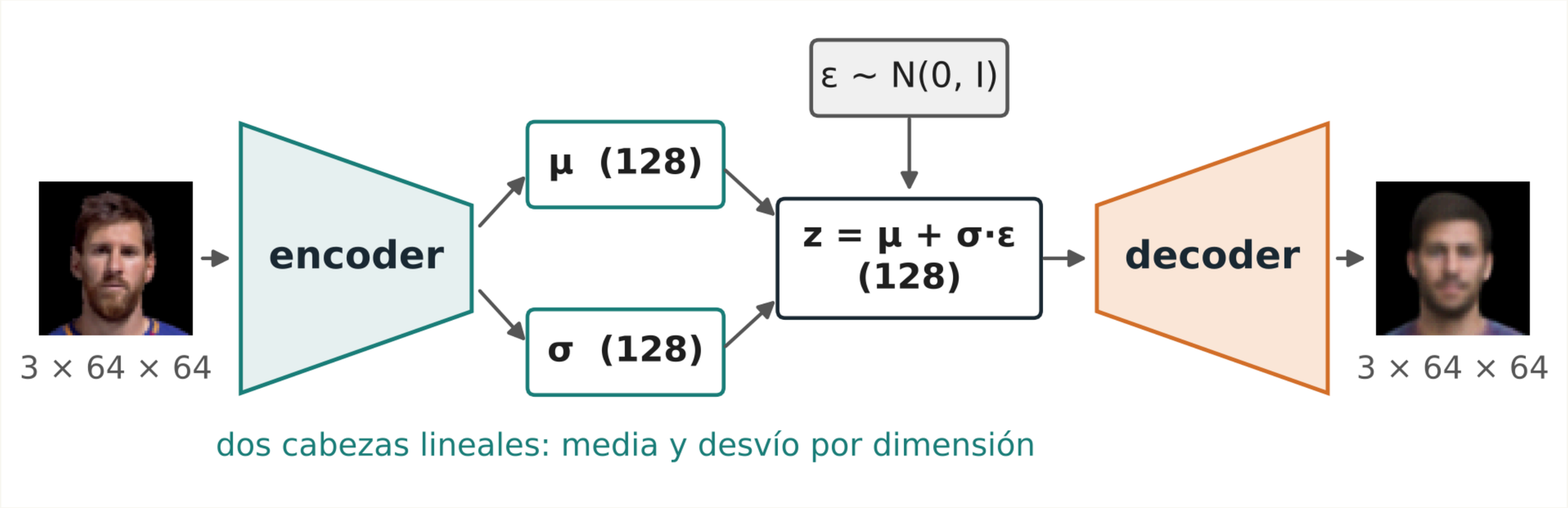
$$z \sim \mathcal{N}(0, I) \quad \longrightarrow \quad \hat{x} = d(z)$$

Sin encoder: z ~ N(0, I) y decodificar.

Mismo experimento que las islas: ahora es **una sola mancha**.

Dígitos en todos lados, con transiciones suaves.

La arquitectura completa



Respecto del autoencoder cambian **dos cosas**: el encoder termina en dos cabezas (μ y σ) y en el medio hay un sorteo reparametrizado. El resto son los mismos bloques.

BLOQUE 5 · VAE SOBRE FIFA

Cuatro detalles de entrenamiento

Latente de 128 dimensiones

128 medias y 128 desvíos por cara. Contra 12.288 píxeles.

Sigmoide + entropía cruzada

Píxeles en [0, 1]: cada uno se trata como una probabilidad, y la reconstrucción se mide píxel por píxel.

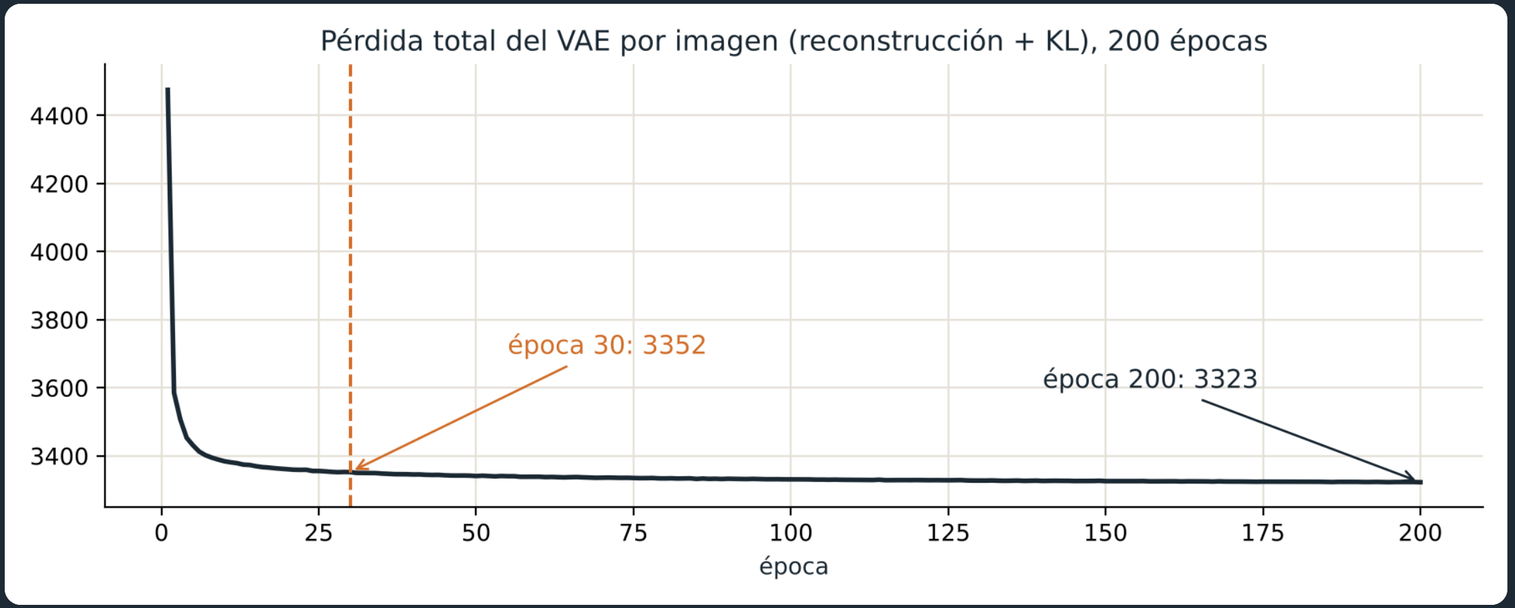
Adam, lr 10^{-3}, batch 128

Sin recetas raras: no hay $\beta_1 = 0{,}5$ ni dos optimizadores. Hay una sola red que optimizar.

200 épocas

El Lab pide 30 (unos minutos en GPU). Las caras de este bloque son de una corrida de 200, sobre ~15.000 retratos.

Esta vez la pérdida sí baja



POR IMAGEN, 200 ÉPOCAS

4475 → 3323

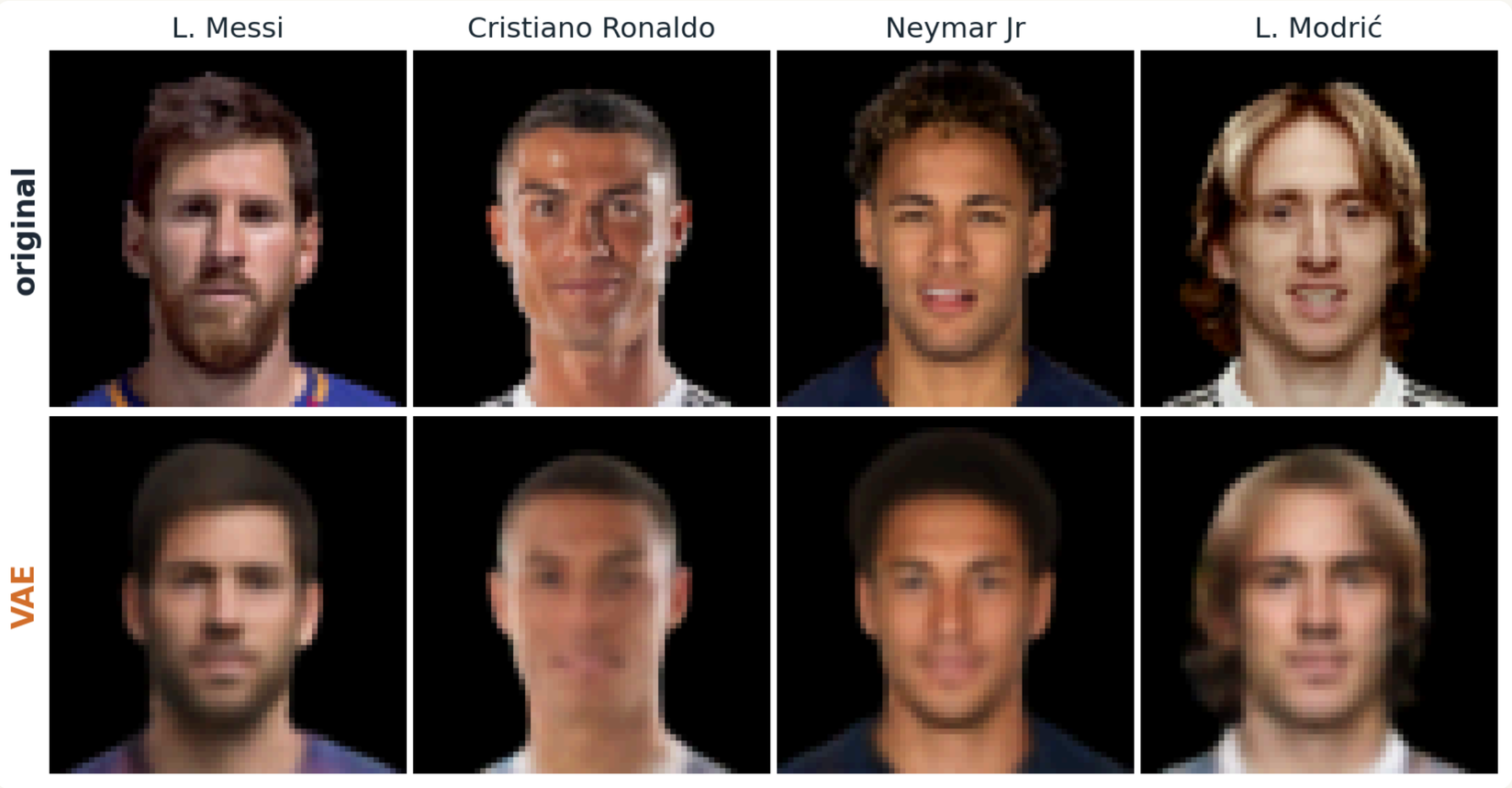
En la época 30 ya estaba en 3352: casi todo lo que baja, baja al principio.

En GANs, la pérdida no informaba. Acá hay **una sola pérdida que se minimiza**, sin rival, y baja de forma monótona.

Aun así: hay que mirar las muestras. Una pérdida baja no garantiza caras nítidas.

BLOQUE 5 · VAE SOBRE FIFA

Reconstrucción



Se reconoce a cada uno: el pelo de Modrić, la sonrisa de Neymar.

Pero todo está **lavado**: sin barba, sin rulos, sin brillo en los ojos.

VAE · 200 ÉPOCAS

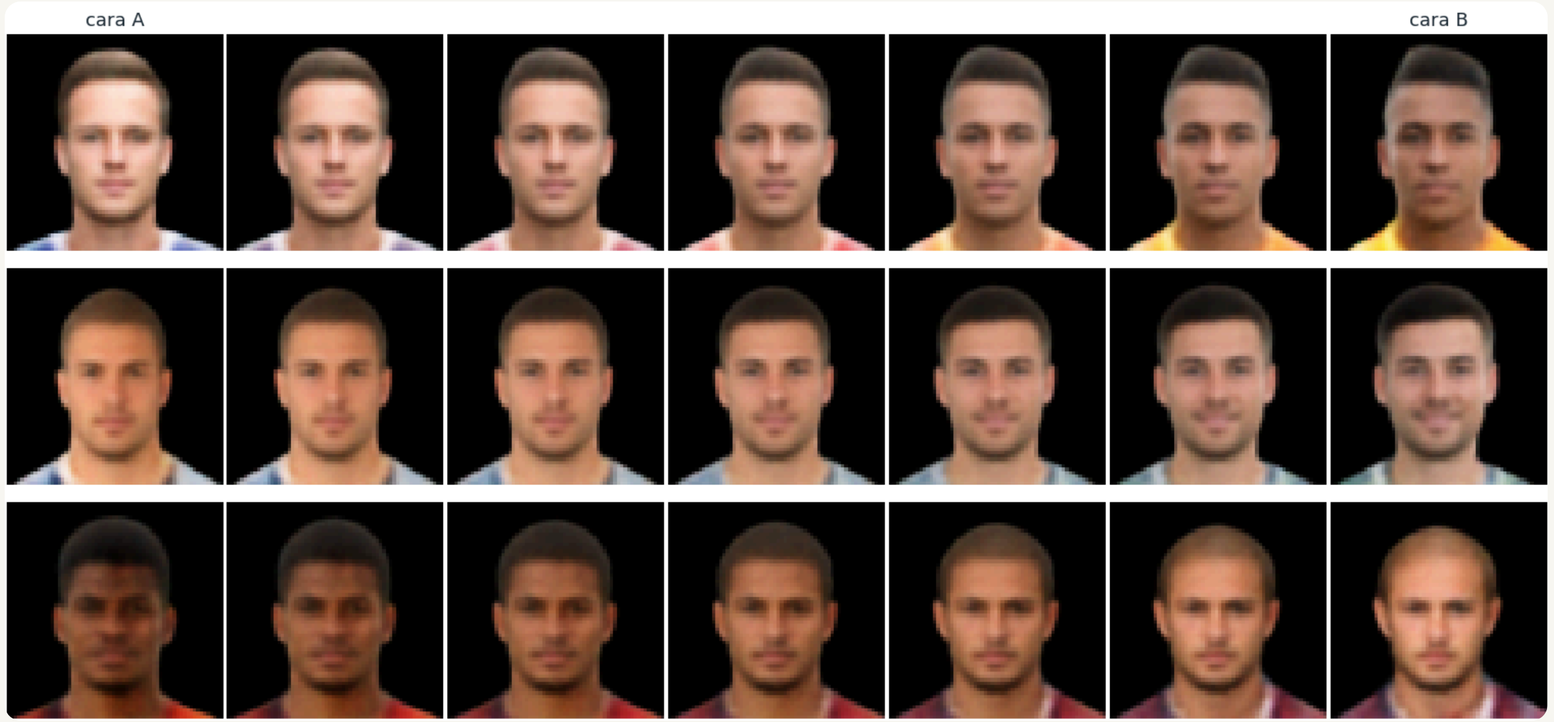
BLOQUE 5 · VAE SOBRE FIFA · 200 ÉPOCAS

Caras nuevas desde el prior: solo z ~ N(0, I)



BLOQUE 5 · VAE SOBRE FIFA · 200 ÉPOCAS

Caminar entre dos caras inventadas



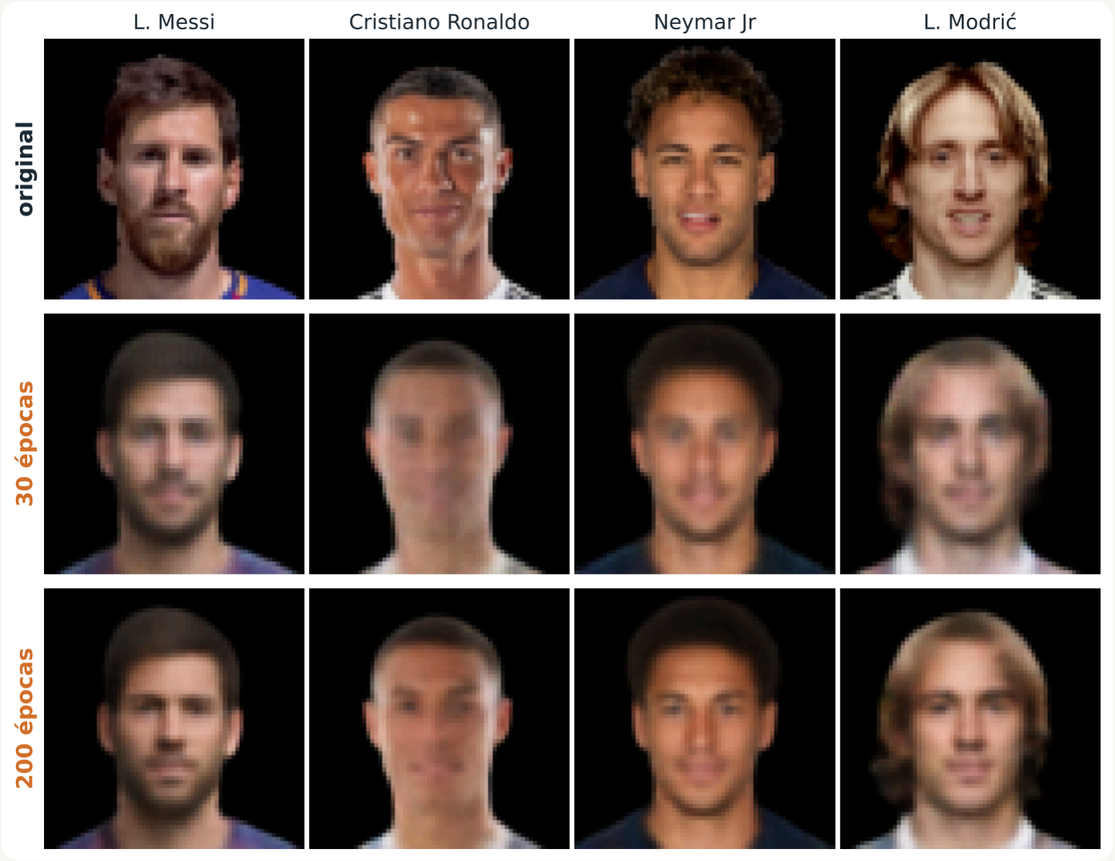
Transición gradual

A y B salen de N(0, I): ninguna es real. En el camino cambian pelo, piel y camiseta, y no aparece basura.

Por diseño

Es la **continuidad** que pedíamos en la slide 13. En un autoencoder, el camino podía cruzar un hueco.

¿Y si entrenamos más?



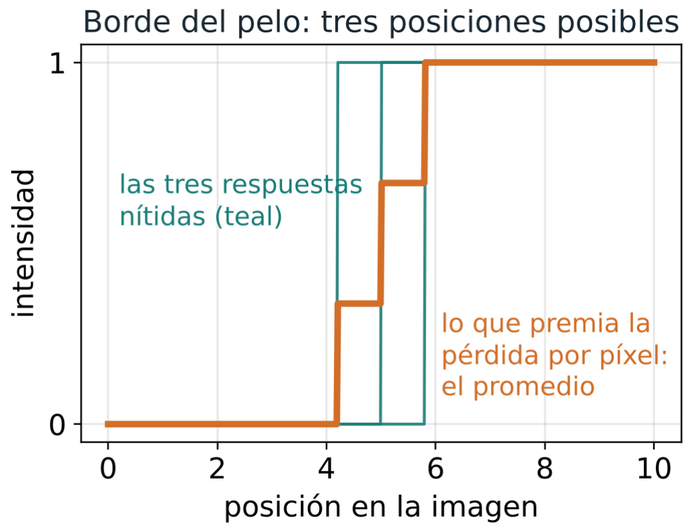
Pérdida por imagen	30 ép.	200 ép.
Total final	3351,5	3322,5
Mejora	—	−0,9 %

La pérdida casi no se mueve, pero a la vista **se nota**: ojos, cejas y el pelo de Modrić, más definidos.

Y aun así: la barba de Messi y los rulos de Neymar **no vuelven**.

> Entrenar más ayuda. Pero el borroso que queda no es falta de entrenamiento: **viene de otro lado**.

Por qué el VAE es borroso







VAE



DCGAN

INTUICIÓN

Si el pelo podría estar un poco más a la izquierda o a la derecha, la pérdida por píxel premia **el promedio** de las caras posibles: un pelo difuso.

La GAN no promedia: el discriminador castiga cualquier cosa que no parezca real, y una cara promedio **no parece real**.

Dos compromisos distintos

	VAE	GAN
Entrenamiento	estable, una pérdida	inestable, un juego entre dos
¿La pérdida informa?	sí, baja de forma monótona	no, oscila
¿Tiene encoder?	sí: ubica una cara real en el latente	no
Nitidez	borroso	nítido
Cobertura	cubre toda la distribución	riesgo de *mode collapse*

No hay ganador: son compromisos distintos. Y hoy existe una tercera vía: **difusión**.

BLOQUE 7 · CIERRE · 2 MINUTOS

En cinco líneas

1. Un autoencoder comprime y descomprime; **el dato es su propia etiqueta**.
2. Un autoencoder no genera: su latente tiene **islas y huecos**.
3. El VAE codifica **nubes, no puntos**, y la KL las ata a N(0, I): un latente continuo y completo.
4. Pérdida = reconstrucción + KL; la reparametrización (**$z = \mu + \sigma \cdot \varepsilon$**) permite entrenar con gradiente.
5. Frente a la GAN: **estable e interpretable, pero borroso**.

BLOQUE 7 · CIERRE

Referencias y lectura

- Kingma y Welling (2013). *Auto-Encoding Variational Bayes*. arXiv:1312.6114.
- Rocca (2019). *Understanding Variational Autoencoders (VAEs)*. Towards Data Science.
- Notebook de la cátedra: Humai, *VAEs.ipynb* en Colab.
- Laboratorio 5b: VAE sobre caras de FIFA.